

Drydock — Launch Distribution Plan

Strategy: publish the authoritative materials once, in a few durable homes, then seed *text-first, link-second* into communities that punish self-promotion. One canonical write-up is the hub; everything else points to it. Lead with the problem; the video and white paper are supporting artifacts, not the pitch.

The map has three layers:

1. **Primary homes** — where the authoritative materials live (repo, white paper, deck, video, docs).
2. **Drop-a-link venues** — where you post a short message with links to redirect interest.
3. **Playbook + sample messages** — how to post cleanly without getting throttled.

1. Primary homes (host the authoritative materials)

Publish the real artifacts here. Each gives a stable, citable URL the drop-a-link posts can point at.

Home	What to put there	Notes
GitHub repo	The methodology + CLI: README as the canonical pitch, docs/ , the deck, install one-liner, 60-second example.	This is the destination for most technical audiences — they click code before video. Make the README the single source of truth.
GitHub Pages	deck.html (self-paced slides) and the rendered white paper / methodology docs.	Free hosting tied to the repo; link from the README.
Zenodo	The white paper (PDF) — gets a permanent DOI and is archival.	Best free path to a citable, versioned white paper. Mint a DOI per release.
arXiv (cs.SE)	A formal methodology paper, if you write one.	Adds academic credibility for the SDD/methodology crowd; may need an endorsement for a first cs.SE submission. Plan lead time.
SSRN	A methodology/working-paper version aimed at the quant/financial-engineering angle.	Reaches the "reproducibility = correctness" audience you speak to natively. Good cross-over home for the financial-engineer framing.
Read the Docs or GitBook	The actual CLI/methodology documentation (commands, SAIL, Rigging).	Searchable, versioned docs that outrank forum threads over time.
dev.to (canonical)	A long-form written version of the talk/white paper, with the video embedded and canonical_url set to your own site if you have one.	SEO + a single citable URL everyone else links to. Set canonical so reposts don't split ranking.

Home	What to put there	Notes
Personal blog / Substack / Ghost	The hub essay and any future deep-dives; collect email subscribers.	Owned channel — the one audience platforms can't throttle. Worth standing up even minimally.
YouTube (unlisted → public)	The recorded talk. Title: Drydock: specification-driven delivery with a real Agile process . Install line + repo link + Q&A in the description and a pinned comment; chapters from script.md timings.	Upload unlisted, do a timing/audio pass with 2–3 people, then flip public. YouTube distrusts polished reels — let the repo carry the weight.
Product Hunt	A launch listing on go-live day: tagline, gallery (deck stills + 30s clip), maker comment.	One-shot attention spike; schedule it, don't waste it. Line up a few people to be present in the comments.

Write the white paper. A 4–8 page PDF — *the missing process in spec-driven development, and why it's Agile* — is the single highest-leverage primary asset besides the repo. It travels into archives (Zenodo/SSRN/arXiv), gets cited, and gives every drop-a-link post something substantive to point at. Mirror it as HTML on GitHub Pages.

2. Drop-a-link venues (short message + links)

Post a short message — "I built a new, functional SDD method; it adds the Agile process spec-driven design was missing — here's the repo / paper / 10-min talk." — and link back to a primary home. Rank by signal, not reach.

Don't blast simultaneously; stagger over 1–2 weeks.

Technical communities (highest signal)

Venue	How to post cleanly	Notes
Hacker News (Show HN)	Show HN: Drydock – governed, spec-driven builds with an Agile process, on your Claude/Codex CLI . Link the repo . First comment = you, plain-text: the problem, what's different, what's rough, what feedback you want.	HN distrusts polish. Repo + honest "here's what's half-built" beats a launch reel. Post 8–10am ET, weekday.
Lobsters	Tags practices / ai . Link the repo or the white paper, not YouTube.	Invite-only, high signal, allergic to marketing. Strongest fit for the methodology crowd.
Reddit — r/programming, r/ExperiencedDevs, r/SoftwareEngineering, r/devtools,	Text post framing the <i>idea</i> (Agile process + spec-as-truth); links second.	r/programming and the github/opensource subs tolerate links; the others

Venue	How to post cleanly	Notes
r/LLMDevs, r/ChatGPTCoding, r/opensource, r/coolgithubprojects		want discussion. Don't cross-post identically same-day.
Lemmy (programming.dev, lemmy.ml/c/programming)	Same framing as Reddit; the fediverse mirror audience.	Smaller but durable, link-friendly.
Echo JS / Designer News	One-line submit with the repo or write-up.	Niche but easy backlinks.
Indie Hackers	A "what I built and why" post; the maker/methodology angle plays well.	Founder audience; good for the product-owner reframing story.

Spec-driven / AI-eng channels (your exact audience)

Venue	How to post cleanly	Notes
GitHub Spec Kit discussions / issues	Engage as a peer: "an alternative take on the build/governance loop; imports Spec Kit projects."	The literal SDD audience. Be complementary, not combative.
Spec-driven / AI-eng Discords & Slacks (Spec Kit Discord, Latent Space, AI-engineering servers)	Drop a one-paragraph "what it is + link" in the relevant channel.	Read each server's self-promo rule first; post where a #show or #projects channel exists.
awesome-spec-driven-development & related awesome-lists	Submit a PR adding Drydock.	Durable backlinks; reaches exactly the searchers you want.

Social (clips + thread)

Venue	How to post cleanly	Notes
X / Mastodon (fosstodon) / Bluesky	A short thread: the problem, the one-idea insight, a 30s clip, link last.	Clips of your bold lines (Slides 2, 8, 13) are the shareable units. Fosstodon is the FOSS-native home.
LinkedIn	The product-owner / process framing as a professional post; link the white paper.	Best venue for the "25 years of best practices" / financial-engineer angle.

Software directories & newsletters (passive discovery)

Venue	How to post cleanly	Notes
AlternativeTo / LibHunt / SaaSHub / Slant / StackShare	List Drydock as an alternative in the SDD / AI-dev-tools categories.	Passive, long-tail discovery; set-and-forget.
FSF Free Software Directory	Submit if the license qualifies.	Signals genuine FOSS, not a SaaS funnel.
Newsletters — Console.dev, TLDR, Pointer.io, The Changelog (news/podcast), Hacker Newsletter	Submit the repo/write-up via their tip forms.	Curated reach into exactly the dev audience; one good pickup outperforms a dozen forum posts.

3. The clean-launch playbook

1. **One write-up is the hub.** A single README or dev.to post is the source of truth; everything else links to it. Don't fragment the message.
2. **Lead with the problem, not the product.** "Spec-driven design skipped the process that ships software — here's the Agile answer" travels further than "I built a tool."
3. **Ship the honest state.** These forums reward "v0, here's what works and what doesn't" over a glossy reel. Your spec already carries **TODO:s** — say so.
4. **Be present for the first 48h.** The launch is the comment thread, not the upload. Answer every reply fast; that's what converts skeptics.
5. **Don't blast simultaneously.** Stagger: HN/Lobsters day 1, Reddit day 2–3, Discords/awesome-lists/directories ongoing. Same-hour cross-posting reads as spam and gets throttled.
6. **Make the repo the destination.** Most audiences click code before video. README must have: the one-idea pitch, the install line, a 60-second example, and the deck/paper/video links.

Sample drop messages

- **Short (Reddit/Discord/social):**

I built a functional spec-driven-development method that adds the part SDD always skipped — the *process*. It's Agile: you're the Commander (product owner), the LLM is your delivery team, and a dependency-graph build keeps the spec and code in sync so you can actually reproduce a build. Runs on your Claude/Codex subscription, no API keys. Repo + 10-min walkthrough:

[<link>](#)

- **Show HN first comment:**

Author here. SDD gave us specs but threw away 25 years of software practice for shipping them — there's no process, so you can't reproduce a build. Drydock adds an Agile loop: analyze → plan a dependency graph → build one context-budgeted step at a time → score drift → refit. It runs on the Claude/Codex CLI (no API keys) and imports Spec Kit projects. It's v0 and rough in places (the spec has open **TODO:s**). I'd love feedback on the process model and the QuarterDeck review flow. [<repo link>](#)

- **One-liner (directories / awesome-list PR):**

Drydock — governed, specification-driven software delivery with an Agile process loop, on your existing Claude/Codex CLI.

4. Pre-publish checklist

- ☐ Repo public; README leads with the insight + install one-liner + 60-second example.
- ☐ White paper published (Zenodo DOI; mirror as HTML on GitHub Pages); SSRN/arXiv if pursuing.
- ☐ `deck.html` on GitHub Pages; linked in README, white paper, and video description.
- ☐ YouTube chapters from `script.md` timings; pinned Q&A comment; repo + install line in description.
- ☐ A single canonical write-up (README or dev.to) all posts point to; `canonical_url` set on any reposts.
- ☐ Replace the `github.com/<your-handle>` placeholder on Slide 14 and everywhere it appears.
- ☐ Sample drop messages adapted per venue; posting order staggered on a calendar.